

LAPORAN PRAKTIKUM

STRUKTUR DATA

“STACK”



Disusun oleh:

Ndaru Pradana Gatot Suseno

2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Jovantri Immanuel Gulo

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya Laporan Praktikum Struktur Data 3 dapat diselesaikan. Laporan ini disusun sebagai dokumentasi kegiatan praktikum sekaligus sarana untuk memperkuat pemahaman mengenai struktur data stack dalam pemrograman Java.

Praktikum ketiga membahas prinsip Last In First Out melalui implementasi stack menggunakan array, ArrayList, dan class Stack pada Java. Pembahasan meliputi operasi push, pop, peek, pencarian nilai maksimum tanpa mengubah susunan stack, serta evaluasi ekspresi postfix. Pada laporan tugas, konsep stack diterapkan untuk membangun simulasi riwayat penjelajahan browser.

Penulis menyampaikan terima kasih kepada dosen pengampu dan asisten praktikum atas arahan selama kegiatan berlangsung. Seluruh uraian disusun berdasarkan source code pada package pekan3_2411532007 serta hasil pengujian program secara langsung. Kritik dan saran yang membangun diharapkan untuk meningkatkan kualitas laporan pada praktikum berikutnya.

Padang, 2026

Ndaru Pradana G.S

DAFTAR ISI

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
Latar Belakang	4
Tujuan Praktikum.....	4
Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
Dasar Teori.....	5
Program StackArray_2411532007 dan Driver	6
Program NilaiMaksimum_2411532007 dan StackPostfix_2411532007.....	8
Program SiswaStack_2411532007	10
BAB III KESIMPULAN.....	12
DAFTAR PUSTAKA	13
LAPORAN TUGAS	14
Tujuan Tugas.....	14
Class Website_2411532007	15
Class Browser_2411532007	16
Hasil Eksekusi Laporan Tugas.....	18
Analisis Hasil	18
Kesimpulan Laporan Tugas	19
Saran.....	19

BAB I

PENDAHULUAN

Latar Belakang

Stack merupakan struktur data linear yang mengatur penambahan dan penghapusan elemen pada satu sisi yang disebut top. Elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan, sehingga pola kerjanya dikenal sebagai Last In First Out. Pola ini digunakan pada pemanggilan fungsi, fitur undo, evaluasi ekspresi, dan riwayat navigasi.

Dalam Java, stack dapat dibangun secara manual menggunakan array atau ArrayList, maupun menggunakan class Stack yang tersedia pada pustaka standar. Praktikum ini membandingkan beberapa bentuk implementasi tersebut dan menerapkannya pada kasus nyata berupa riwayat website, sehingga operasi push, pop, dan peek dapat diamati melalui interaksi pengguna.

Tujuan Praktikum

1. Memahami konsep LIFO dan elemen top pada stack.
2. Menerapkan operasi push, pop, peek, isEmpty, dan penelusuran stack.
3. Mengimplementasikan stack menggunakan array, ArrayList, dan java.util.Stack.
4. Menggunakan stack untuk mencari nilai maksimum dan mengevaluasi ekspresi postfix.
5. Menerapkan stack pada simulasi browser history.

Manfaat Praktikum

Praktikum ini melatih kemampuan memilih implementasi stack sesuai kebutuhan, memahami perubahan posisi top, serta mengembangkan program yang memanfaatkan urutan LIFO. Pemahaman tersebut berguna untuk penyelesaian masalah navigasi, pemrosesan ekspresi, backtracking, dan pengelolaan riwayat operasi.

BAB II

PEMBAHASAN

Dasar Teori

Stack memiliki operasi utama push untuk menambahkan data ke top, pop untuk mengambil sekaligus menghapus data teratas, peek untuk membaca data teratas tanpa menghapusnya, dan isEmpty untuk memeriksa kondisi kosong. Implementasi berbasis array menggunakan indeks top, sedangkan implementasi pustaka Java mengelola kapasitas secara dinamis.

Evaluasi postfix memanfaatkan stack dengan memasukkan operand dan mengeluarkan dua operand ketika operator ditemukan. Pada browser history, setiap halaman baru didorong ke stack; tombol back menghapus halaman aktif dan menampilkan halaman sebelumnya melalui peek.

Program StackArray_2411532007 dan Driver

StackArray_2411532007 membangun stack integer dengan array berkapasitas 1000 dan variabel top_2007. StackArrayDriver_2411532007 menguji penambahan tiga nilai, penghapusan elemen teratas, pemeriksaan top, dan penampilan isi stack.

Source Code StackArray_2411532007

```

1 package pekan3_2411532007;
2
3 public class StackArray_2411532007 {
4     static final int MAX_2007 = 1000;
5     int top_2007;
6     int a_2007[] = new int [MAX_2007];
7
8     boolean isEmpty_2007() {return (top_2007 < 0);}
9
10    StackArray_2411532007() {top_2007 = -1 ;}
11
12    boolean push_2007(int x) {
13        if ( top_2007 >= (MAX_2007 - 1)) {
14            System.out.println("Stack Overflow");
15            return false;
16        }
17        else {
18            a_2007[++top_2007] = x;
19            System.out.println(x + "dimasukkan ke dalam stack");
20            return true;
21        }
22    }
23
24    int pop_2007() {
25        if ( top_2007 < 0) {
26            System.out.println("Stack Overflow");
27            return 0;
28        }
29        else {
30            int x_2007 = a_2007[top_2007--];
31            return x_2007;
32        }
33    }
34
35    int peek_2007() {
36        if (top_2007 < 0) {
37            System.out.println("Stack underflow");
38            return 0;
39        }
40        else {
41            int x_2007 = a_2007[top_2007];
42            return x_2007;
43        }
44    }
45
46    void print_2007() {
47        for ( int i = top_2007; i > -1; i--) {
48            System.out.print (" " + a_2007[i]);
49        }
50    }
51 }
52

```

Source Code StackArrayDriver_2411532007

```

1 package pekan3_2411532007;
2
3 public class StackArrayDriver_2411532007 {
4     public static void main (String []args) {
5         StackArray_2411532007 s_2007 = new StackArray_2411532007();
6
7         s_2007.push_2007(10);
8         s_2007.push_2007(20);
9         s_2007.push_2007(30);
10        System.out.println(s_2007.pop_2007() + "dikeluarkan dari stack ");
11        System.out.println("Elemen teratas adalah " + s_2007.peek_2007());
12        System.out.print("Element pada stack");
13        s_2007.print_2007();
14    }
15 }
16

```

Penjelasan Operasi

1. `push_2007()` menaikkan `top` lalu menyimpan nilai pada array.
2. `pop_2007()` mengembalikan nilai pada `top` dan menurunkan indeks `top`.
3. `peek_2007()` membaca elemen teratas tanpa menghapusnya.
4. `print_2007()` menampilkan data dari posisi `top` menuju dasar stack.

Hasil Eksekusi

```
10dimasukkan ke dalam stack
20dimasukkan ke dalam stack
30dimasukkan ke dalam stack
30dikeluarkan dari stack
Elemen teratas adalah 20
Element pada stack 20 10
```

Analisis

Nilai 10, 20, dan 30 dimasukkan secara berurutan sehingga 30 berada pada `top`. Operasi `pop` mengeluarkan 30, kemudian `peek` menghasilkan 20. Penampilan akhir menunjukkan urutan 20 dan 10 dari `top` menuju dasar stack.

Pada pesan keluaran terdapat kata yang menyatu seperti “10dimasukkan”. Hal tersebut hanya berkaitan dengan format String dan tidak mengubah perilaku stack.

Program NilaiMaksimum_2411532007 dan StackPostfix_2411532007

Dua program ini menunjukkan pemanfaatan `java.util.Stack` untuk operasi pemrosesan data. `NilaiMaksimum` mencari nilai terbesar dengan stack cadangan, sedangkan `StackPostfix` mengevaluasi ekspresi postfix.

Source Code NilaiMaksimum_2411532007

```
1 package pekan3_2411532007;
2 import java.util.Stack;
3
4
5
6 public class NilaiMaksimum_2411532007 {
7     public static int max_2007(Stack<Integer> s_2007) {
8         Stack<Integer> backup_2007 = new Stack<Integer>();
9         int maxValue_2007 = s_2007.pop();
10        backup_2007.push(maxValue_2007);
11
12        while (!s_2007.isEmpty()) {
13            int next_2007 = s_2007.pop();
14            backup_2007.push(next_2007);
15            maxValue_2007 = Math.max(maxValue_2007, next_2007);
16        }
17
18        while (!backup_2007.isEmpty()) {
19            s_2007.push(backup_2007.pop());
20        }
21        return maxValue_2007;
22    }
23
24    public static void main (String []args) {
25        Stack<Integer> s_2007 = new Stack<Integer>();
26        s_2007.push(70);
27        s_2007.push(12);
28        s_2007.push(20);
29
30        System.out.println("Isi Stack " + s_2007);
31        System.out.println("Stack teratas " + s_2007.peek());
32        System.out.println("Nilai teratas " + max_2007(s_2007));
33    }
34 }
35
36
```

Source Code StackPostfix_2411532007

```
1 package pekan3_2411532007;
2 import java.util.Stack;
3
4
5 public class StackPostfix_2411532007 {
6     public static int postfixEvaluate_2007(String expression) {
7         Stack<Integer> s_2007 = new Stack<Integer>();
8         Scanner input_2007 = new Scanner(expression);
9
10        while (input_2007.hasNext()) {
11
12            if (input_2007.hasNextInt()) {
13                s_2007.push(input_2007.nextInt());
14            }
15            else {
16                String operator_2007 = input_2007.next();
17                int operand2_2007 = s_2007.pop();
18                int operand1_2007 = s_2007.pop();
19
20                if (operator_2007.equals("+")) {
21                    s_2007.push(operand1_2007 + operand2_2007);
22                } else if (operator_2007.equals("-")) {
23                    s_2007.push(operand1_2007 - operand2_2007);
24                } else if (operator_2007.equals("*")) {
25                    s_2007.push(operand1_2007 * operand2_2007);
26                } else if (operator_2007.equals("/")) {
27                    s_2007.push(operand1_2007 / operand2_2007);
28                }
29            }
30        }
31
32        input_2007.close();
33        return s_2007.pop();
34    }
35
36    public static void main(String [] args) {
37        System.out.println("Hasil postfix= " + postfixEvaluate_2007("5 2 4 * + 7 -"));
38    }
39 }
40
```

Hasil Eksekusi


```
isi Stack [70, 12, 20]  
Stack teratas 20  
Nilai teratas 70
```

Analisis

Method `max_2007` memindahkan seluruh elemen ke stack backup sambil memperbarui `maxValue_2007`. Setelah itu, elemen dikembalikan sehingga susunan stack awal tetap dipertahankan. Dari data `[70, 12, 20]`, nilai maksimum yang diperoleh adalah 70.

Ekspresi postfix “5 2 4 * + 7 -” dihitung menjadi 6. Angka dimasukkan ke stack, sedangkan setiap operator mengambil dua operand sesuai urutan operand1 operator operand2.

Program SiswaStack_2411532007

Program ini menggunakan ArrayList sebagai penyimpanan internal stack objek Siswa_2007. Elemen ditambahkan pada akhir list dan dihapus dari indeks terakhir.

Source Code SiswaStack_2411532007

```

1 package pekan3_2411532007;
2 import java.util.ArrayList;
3
4 class Siswa_2007 {
5     String nama_2007;
6     int nim_2007;
7
8     public Siswa_2007(String nama, int nim) {
9         this.nama_2007 = nama;
10        this.nim_2007 = nim;
11    }
12
13    public String toString() {
14        return "Nim: " + nim_2007 + ", Nama: " + nama_2007;
15    }
16 }
17
18 public class SiswaStack_2411532007 {
19     private ArrayList<Siswa_2007> stack_2007;
20
21     public SiswaStack_2411532007() {
22         stack_2007 = new ArrayList<>();
23     }
24
25     public void push_2007(Siswa_2007 mhs) {
26         stack_2007.add(mhs);
27     }
28
29     public Siswa_2007 pop_2007() {
30         if (!isEmpty_2007()) {
31             return stack_2007.remove(stack_2007.size()-1);
32         }
33         return null;
34     }
35
36     public Siswa_2007 peek_2007() {
37         if (!isEmpty_2007()) {
38             return stack_2007.get(stack_2007.size() - 1);
39         }
40         return null;
41     }
42
43     public boolean isEmpty_2007() {
44         return stack_2007.isEmpty();
45     }
46
47     public void tampilkanSiswa_2007() {
48         for (int i = stack_2007.size() - 1; i >= 0; i--) {
49             System.out.println(stack_2007.get(i));
50         }
51     }
52
53     public static void main (String [] args) {
54         SiswaStack_2411532007 studentStack_2007 = new SiswaStack_2411532007();
55
56         Siswa_2007 mhs1 = new Siswa_2007("Ali", 1);
57         Siswa_2007 mhs2 = new Siswa_2007("Boby", 2);
58         Siswa_2007 mhs3 = new Siswa_2007("charles", 3);
59
60         studentStack_2007.push_2007(mhs1);
61         studentStack_2007.push_2007(mhs2);
62         studentStack_2007.push_2007(mhs3);
63
64         System.out.println("Siswa di dalam stack:");
65         studentStack_2007.tampilkanSiswa_2007();
66
67         System.out.println("Siswa teratas " + studentStack_2007.peek_2007());
68         System.out.println("Mengeluarkan siswa teratas dari stack" + studentStack_2007.pop_2007());
69         System.out.println("daftar siswa setelah di pop :");
70         studentStack_2007.tampilkanSiswa_2007();
71     }
72 }
73
74

```

Penjelasan Operasi

1. push_2007() menambahkan objek siswa ke bagian akhir ArrayList.
2. pop_2007() menghapus objek pada indeks size - 1.

3. peek_2007() membaca objek terakhir tanpa menghapusnya.
4. tampilkanSiswa_2007() menelusuri data dari indeks terakhir ke indeks pertama.

Hasil Eksekusi

```
Siswa di dalam stack:  
Nim: 3, Nama: charles  
Nim: 2, Nama: Bobby  
Nim: 1, Nama: Ali  
Siswa teratas Nim: 3, Nama: charles  
Mengeluarkan siswa teratas dari stackNim: 3, Nama: charles  
daftar siswa setelah di pop :  
Nim: 2, Nama: Bobby  
Nim: 1, Nama: Ali
```

Analisis

Ali, Bobby, dan Charles dimasukkan secara berurutan. Karena Charles dimasukkan terakhir, objek tersebut tampil sebagai elemen teratas dan menjadi objek pertama yang dikeluarkan ketika pop dipanggil.

Setelah operasi pop, stack hanya berisi Bobby dan Ali. Hasil tersebut menegaskan penerapan prinsip LIFO pada kumpulan objek.

BAB III

KESIMPULAN

Praktikum 3 menunjukkan bahwa stack dapat diimplementasikan menggunakan array, ArrayList, maupun class Stack dengan prinsip operasi yang sama. Perubahan nilai top menentukan elemen yang ditambahkan, dibaca, dan dihapus.

Konsep LIFO berhasil diterapkan pada pengelolaan objek siswa, pencarian nilai maksimum, evaluasi postfix, dan browser history. Setiap implementasi memperlihatkan bahwa data terakhir yang masuk menjadi data pertama yang diproses keluar.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Sierra, K., Bates, B., & Gee, T. (2022). Head First Java (3rd ed.). O'Reilly Media.

Oracle. (2024). Java Platform, Standard Edition API Specification. Oracle Corporation.

LAPORAN TUGAS

Laporan tugas Praktikum 3 menerapkan stack pada simulasi browser history. Website_2411532007 berfungsi sebagai model halaman, sedangkan Browser_2411532007 menyimpan objek halaman pada Stack<Website_2411532007> dan menyediakan operasi kunjungi, kembali, melihat halaman aktif, serta menampilkan seluruh riwayat.

Tujuan Tugas

1. Membuat class Website sebagai representasi judul dan URL.
2. Menyimpan objek website dalam stack.
3. Menerapkan push ketika halaman dikunjungi.
4. Menerapkan pop dan peek pada tombol back serta halaman aktif.
5. Menguji riwayat penjelajahan melalui menu interaktif.

Class Website_2411532007

Class Website_2411532007 menyimpan judul halaman dan URL. Konstruktor menginisialisasi kedua atribut, sedangkan setter dan getter menyediakan akses terhadap data halaman.

Source Code

```
1 package pekan3_2411532007;
2
3 public class Website_2411532007 {
4     private String Judul_2007;
5     private String url_2007;
6
7     public Website_2411532007(String j, String u) {
8         this.Judul_2007 = j;
9         this.url_2007 = u;
10    }
11
12    public void setJ_2007(String j) {
13        this.Judul_2007 = j;
14    }
15
16    public void setU_2007(String u) {
17        this.url_2007 = u;
18    }
19
20    public String getJ_2007() {
21        return Judul_2007;
22    }
23
24    public String getUrl_2007() {
25        return url_2007;
26    }
27
28 }
29 }
```

Penjelasan

1. Judul_2007 menyimpan nama halaman website.
2. url_2007 menyimpan alamat halaman.
3. getJ_2007() dan getUrl_2007() digunakan Browser untuk menampilkan informasi halaman.

Class Browser_2411532007

Class Browser_2411532007 menjadi pengendali program sekaligus pengelola stack history. Setiap halaman baru didorong ke stack, sementara tombol back menghapus halaman aktif jika masih terdapat halaman sebelumnya.

Source Code

```

1 package pekan3_2411532007;
2
3 import java.util.Scanner;
4
5
6 public class Browser_2411532007 {
7     private Stack<Website_2411532007> history_2007 = new Stack<>();
8     private Scanner input = new Scanner(System.in);
9
10    public void KunjungiWeb_2007(Website_2411532007 w) {
11        history_2007.push(w);
12    }
13
14    public void back_2007() {
15        if (history_2007.isEmpty()) {
16            System.out.println("\nStack kosong. Tidak ada halaman untuk dikembalikan.\n");
17            return;
18        }
19
20        if (history_2007.size() == 1) {
21            System.out.println("\nTidak ada halaman sebelumnya. Anda sudah di halaman pertama.\n");
22            return;
23        }
24
25        history_2007.pop();
26
27        Website_2411532007 a_2007 = history_2007.peek();
28
29        System.out.println("\nKembali ke page: " + a_2007.getJ_2007() + "\n");
30    }
31
32    public void LihatHalamanSaatIni_2007() {
33        if (history_2007.isEmpty()) {
34            System.out.println("\nTidak ada halaman yang sedang dibuka.\n");
35            return;
36        }
37
38        Website_2411532007 b_2007 = history_2007.peek();
39        System.out.println("\nHalaman saat ini: " + b_2007.getJ_2007());
40        System.out.println("URL: " + b_2007.getUrl_2007() + "\n");
41    }
42
43    public void tampilkanHistory_2007() {
44        System.out.println("\n=== Riwayat Penjelajahan (History) ===");
45        System.out.println("\n=== Riwayat Penjelajahan (History) ===");
46
47        if (history_2007.isEmpty()) {
48            System.out.println("History kosong. Belum ada website yang dikunjungi.\n");
49            return;
50        }
51
52        System.out.println("Total halaman dalam history: " + history_2007.size());
53        System.out.println("-----");
54
55        int no = 1;
56
57        for (Website_2411532007 website : history_2007) {
58            System.out.println(no + ". " + website.getJ_2007());
59            System.out.println("    URL: " + website.getUrl_2007());
60            System.out.println();
61            no++;
62        }
63
64        System.out.println("-----\n");
65    }
66
67
68    public void tampilkanMenu() {
69        System.out.println("=== Browser History NIM: 2411532007 ===\n");
70        System.out.println("1. Kunjungi Website (Push)");
71        System.out.println("2. Tombol Back (Pop)");
72        System.out.println("3. Lihat Halaman Aktif (Peek)");
73        System.out.println("4. Tampilkan History");
74        System.out.println("5. Keluar");
75        System.out.print("Pilihan: ");
76    }
77
78
79    public void jalankan() {
80        boolean berjalan_2007 = true;
81
82        while (berjalan_2007) {
83            tampilkanMenu();
84            int pilihan_2007 = input.nextInt();
85            input.nextLine();
86

```



```

86         input.nextLine();
87
88         switch (pilihan_2007) {
89             case 1:
90                 System.out.print("\nMasukkan Judul: ");
91                 String judul = input.nextLine();
92
93                 System.out.print("Masukkan URL: ");
94                 String url = input.nextLine();
95
96                 Website_2411532007 website = new Website_2411532007(judul, url);
97                 KunjungiWeb_2007(website);
98
99                 System.out.println("\nBerhasil mengunjungi halaman!\n");
100                break;
101
102                case 2:
103                    back_2007();
104                    break;
105
106                case 3:
107                    LihatHalamanSaatIni_2007();
108                    break;
109
110                case 4:
111                    tampilkanHistory_2007();
112                    break;
113
114                case 5:
115                    System.out.println("\nTerima kasih telah menggunakan browser ini!");
116                    berjalan_2007 = false;
117                    break;
118
119                default:
120                    System.out.println("\nPilihan tidak valid. Silakan coba lagi.\n");
121                    break;
122            }
123        }
124        input.close();
125    }
126
127
128
129
130    public static void main(String[] args) {
131        Browser_2411532007 browser_2007 = new Browser_2411532007();
132        browser_2007.jalankan();
133    }
134 }
135

```

Penjelasan

1. KunjungiWeb_2007() menambahkan objek Website ke stack melalui push().
2. back_2007() memvalidasi ukuran stack, melakukan pop, lalu menampilkan halaman sebelumnya dengan peek().
3. LihatHalamanSaatIni_2007() membaca elemen top tanpa menghapusnya.
4. tampilkanHistory_2007() menelusuri seluruh objek yang tersimpan.
5. jalankan() mengatur menu interaktif sampai pengguna memilih keluar.

Hasil Eksekusi Laporan Tugas

```

=== Browser History NIM: 2411532007 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Tampilkan History
5. Keluar
Pilihan: 1

Masukkan Judul: google
Masukkan URL: http://google.com

Berhasil mengunjungi halaman!

=== Browser History NIM: 2411532007 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Tampilkan History
5. Keluar
Pilihan: 3

Halaman saat ini: google
URL: http://google.com

=== Browser History NIM: 2411532007 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Tampilkan History
5. Keluar
Pilihan: 4

=== Riwayat Penjelajahan (History) ===
Total halaman dalam history: 1
-----
1. google
   URL: http://google.com
-----
1. google
   URL: http://google.com
-----

=== Browser History NIM: 2411532007 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Tampilkan History
5. Keluar
Pilihan: 2

Tidak ada halaman sebelumnya. Anda sudah di halaman pertama.

=== Browser History NIM: 2411532007 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Tampilkan History
5. Keluar
Pilihan: 5

Terima kasih telah menggunakan browser ini!

```

Analisis Hasil

Pengujian mengunjungi halaman Google lalu GitHub. Stack menyimpan Google pada dasar dan GitHub pada top, sehingga menu halaman aktif menampilkan GitHub. Menu history menampilkan dua halaman sesuai urutan kunjungan.

Ketika tombol back dipilih, objek GitHub dikeluarkan melalui pop dan halaman Google menjadi top baru. Pemeriksaan berikutnya menampilkan Google sebagai halaman aktif. Validasi ukuran stack mencegah pengguna kembali ketika hanya tersisa satu halaman.

Kesimpulan Laporan Tugas

Website_2411532007 dan Browser_2411532007 berhasil memodelkan riwayat browser dengan stack. Operasi push, pop, dan peek bekerja sesuai prinsip LIFO serta menghasilkan alur navigasi kembali yang logis.

Saran

Program dapat dikembangkan dengan fitur forward menggunakan stack kedua, penyimpanan waktu kunjungan, penghapusan seluruh history, validasi URL, serta penanganan input nonangka agar aplikasi lebih robust.